

PRACTICAL TASK THREE

DATABASE & FILESERVER CONNECTIVITY

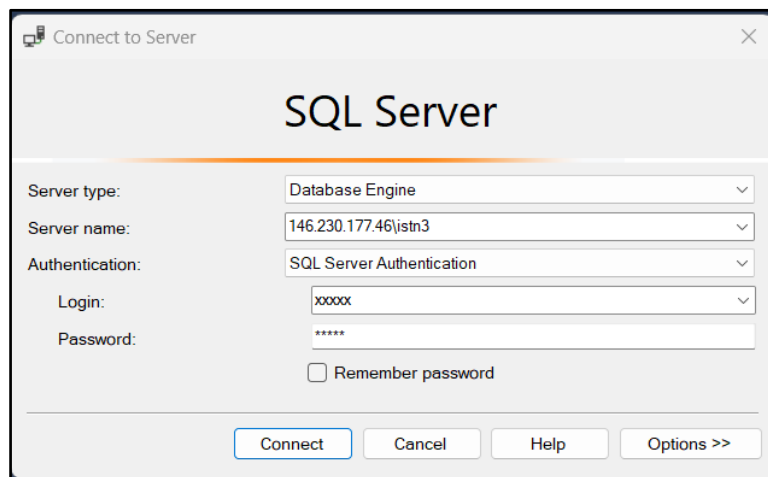
OBJECTIVES

- Establish familiarity with Sql Server development tools
- Test out Sql Server individual accounts on Sql Server
- Create a simple database structures
- Populate database tables with data
- Create a C# application using Visual Studio (VS)
- Establish database connectivity with Sql Server database
- Display data from Sql Server using Windows Forms

PART A

1. Launch Sql Server Management Studio
2. Activate GlobalProtect (University VPN)
3. Establish a connection to the Sql Server
4. Note: The IP address for the SQL Server database is:

146.230.177.46\ISTN3



Navigate to your individual/personal Database

Open up the *tables* tab and add the following new tables

5. Create a Customer table (or Cust table) where the CustID field is the primary key (PK) that is also an auto-increment field starting from 1, incremented by one.

Save the table in Sql Server as “Customer” (or Cust)

WSTISTWEB2.PointOfSale - dbo.Cust			
	Column Name	Data Type	Allow Nulls
🔑	CustId	int	☐
	FirstName	varchar(50)	☐
	Surname	varchar(50)	☐
	email	varchar(50)	☒
	ContactNo	varchar(50)	☒
▶	RewardsBalance	decimal(8, 2)	☒

6. Create an Order table where the **OrderNo is the PK that is also an auto-increment key**

WSTISTWEB2.PointOfSale - dbo.Order			
	Column Name	Data Type	Allow Nulls
🔑	OrderNo	int	☐
	OrderTotal	decimal(6, 2)	☐
	CustId	int	☐
	OrderDate	smalldatetime	☐
	OrderRewards	decimal(8, 2)	☒

Save as “OrderTable”

7. Now add some fictitious data into the tables (as many as you can). However, make sure that for each CustId that is entered in the OrderTable, there is a matching CustId in the Customer table. (Why?). Save both the tables.

In order to add data, this can be initially done via Management Studio. Right click the table and select “Edit top 200 rows”.

	CustId	FirstName	Surname	email	ContactNo	RewardsBa.
▶	1	Reginald	Qwabe	qwabe@ukzn.ac.za	12345	500,00
	2	Miley	Cyrus	cyrus@gmail.com	23456	450,00
	3	Black	Coffee	coffee@yahoo.com	32456	1200,00
	4	Cassper	Nyovest	nyovest@gmail.com	44443	500,00
	5	James	Blunt	blunt@ukzn.ac.za	45632	350,00
	6	Peter	Naidoo	naidoo@ukzn.ac.za	23457	200,00
	7	Pryankra	Chopra	chopra@gmail.com	32467	600,00

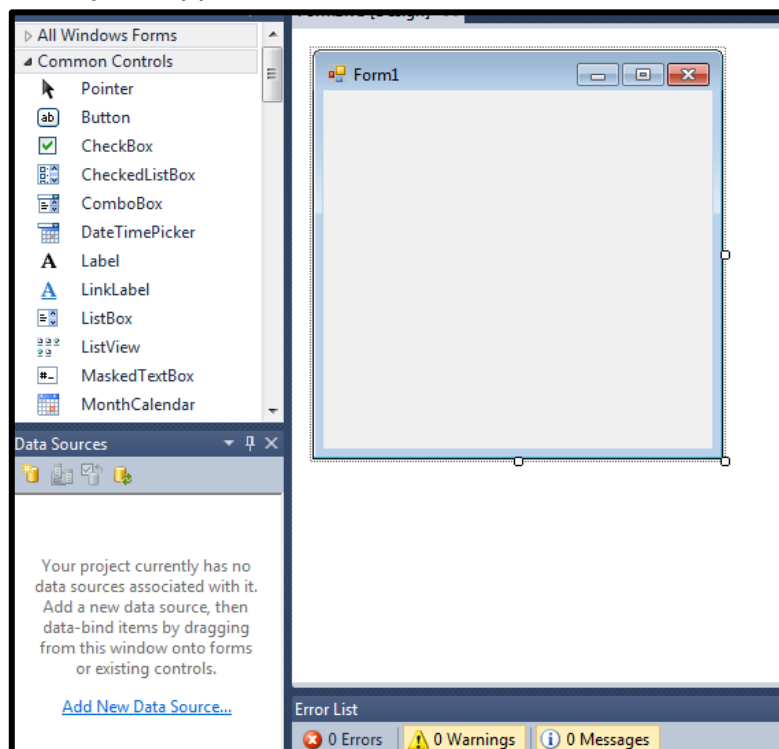
More data rows will be ideal because it will create a “realistic” presentation

	OrderNo	OrderTotal	CustId	OrderDate	OrderRewards
	1	400,00	3	2026-02-14 15:22:00	40,00
	2	456,00	2	2026-03-14 12:24:00	46,00
	3	100,00	3	2026-03-15 13:32:00	55,00
	4	250,00	1	2026-03-16 15:45:00	62,00
	5	400,00	3	2026-03-26 10:18:00	120,00
	6	54,00	3	2026-03-26 11:22:00	84,00
	7	73,00	7	2021-03-27 09:11:00	200,00

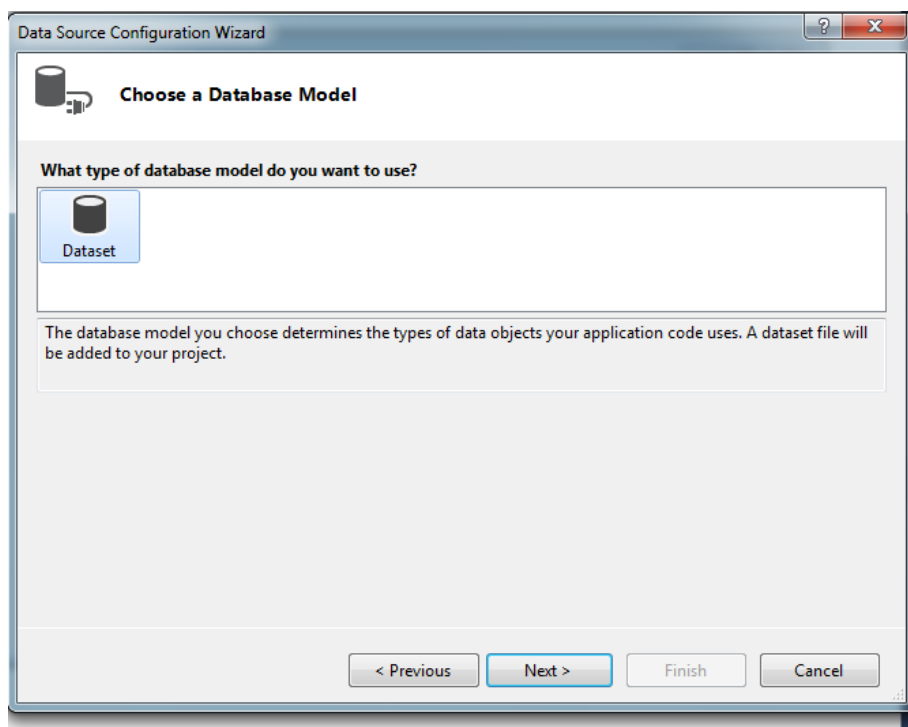
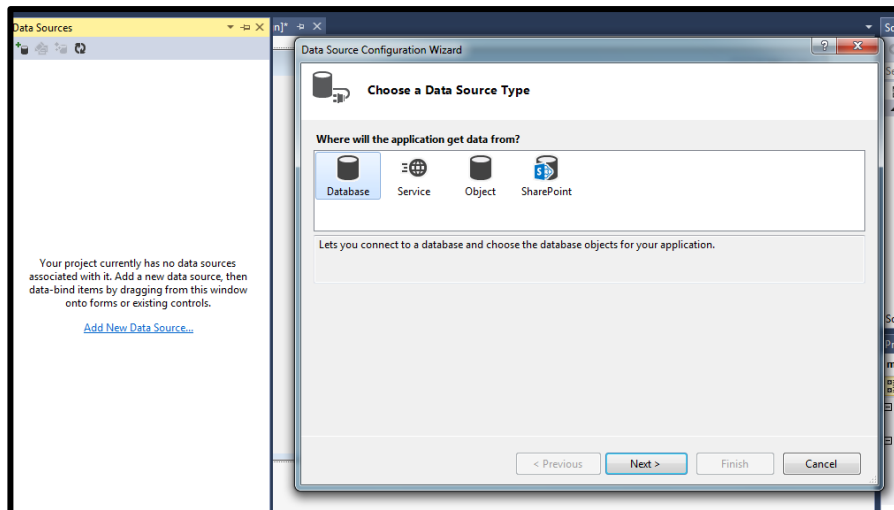
Note: The OrderNo is system generated and will start at 1 by default

PART B

8. Create a new Windows based C# project in Visual Studio. Add a new data source to your application.



9. Add a New Data SourceChoose the options database>>>dataset in response to the prompts from the data connection wizard

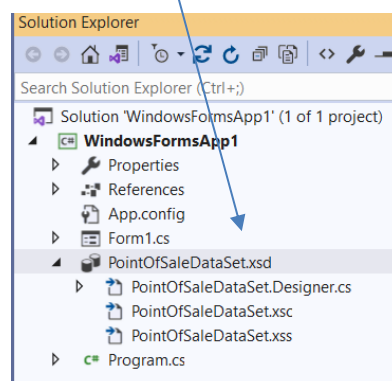


Select New Connection (if this is the first time that you're connecting to the database)

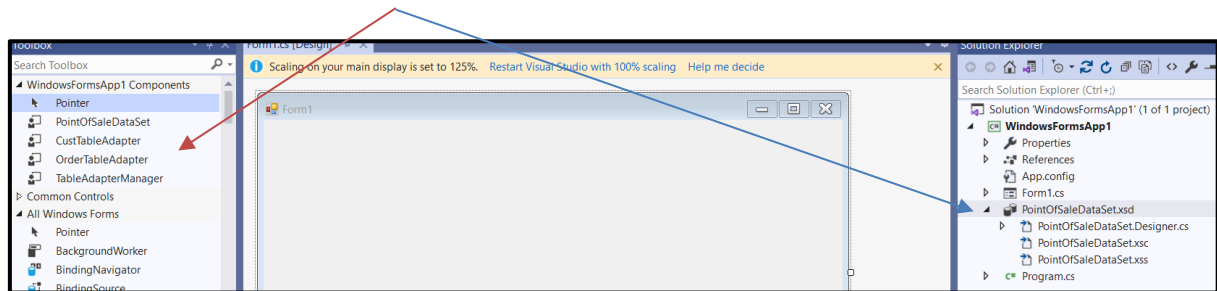
Note: The Database name will be your username!

10. Select both the tables from the dialog box

11. An in-memory copy of the data from the database is created and stored as a class referred to as a dataset. All further manipulations of data will occur via the dataset.



In order to make this class available as an object in the application (so that it can be used), you have to re-build the application. Hereafter, the class becomes available as a toolbox item.

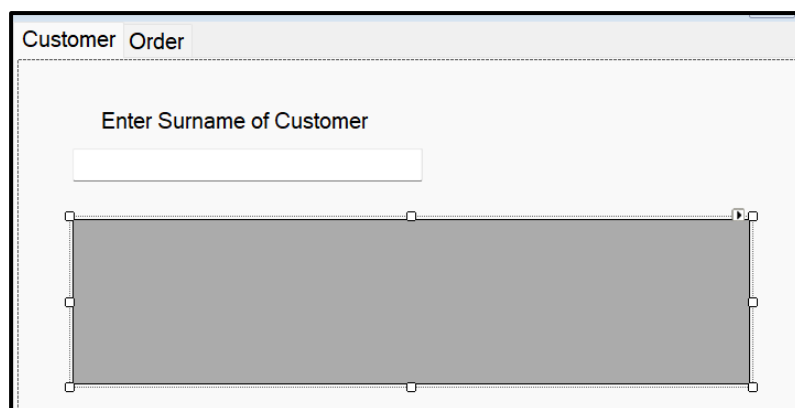


You can now experiment with the data controls that are made available via the interface presented to you.

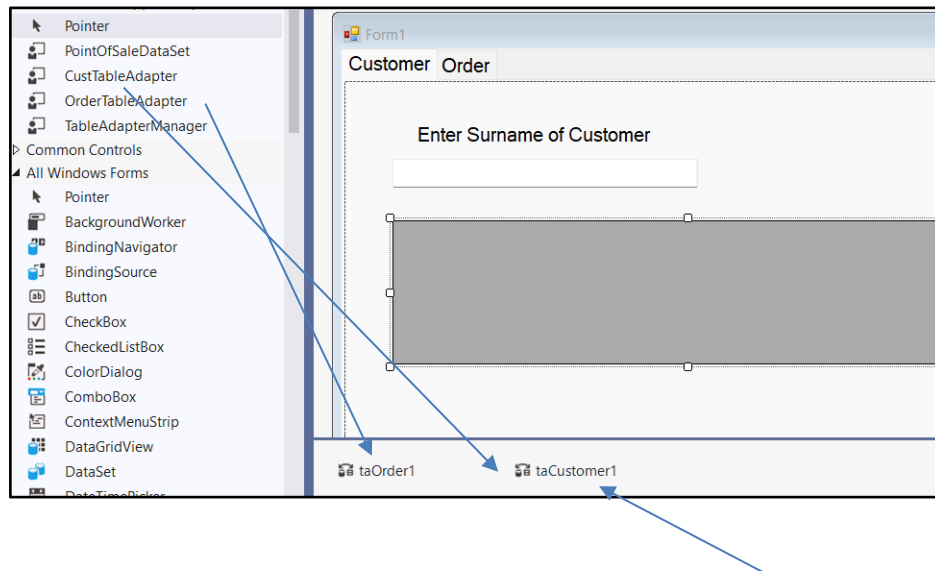
12. For this prac we will **not** create a full-blown menu system. We will however make use of the tab control to present data and functionality on a single form. Add a tab control onto the form and use the ***dock property*** to ***fill*** the entire form with the tab



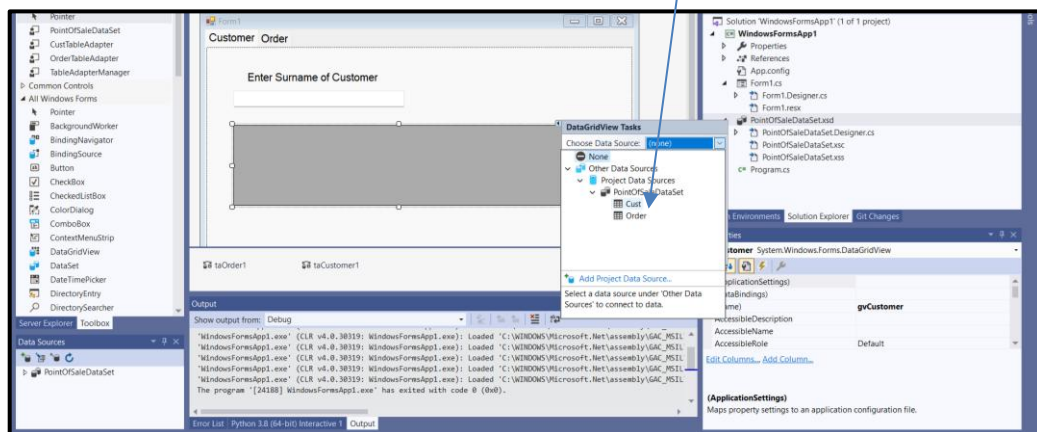
13. The Customer tab will be used to search for a customer. Once the customer is found, all the order entries for that customer will be displayed. The interface to do this will consist of a textbox, label and a gridview control.



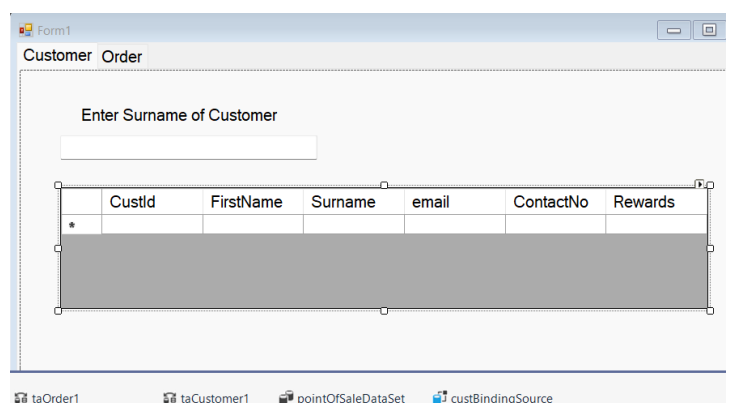
14. You need to set the form up so that the database classes become available as objects on the application. After re-building the application, drag and drop the Customer and Order table adapters from the toolbox into the tray of the form



15. Set the gridview so that the data source is the Customer table adapter object (taCustomer1)



16. VS adds the extra objects (dataset and a bindingsource) automatically. The dataset is more important and you could rename it to be a more “comfortable” name because it will be used quite frequently in the processing (something like **ds1**).

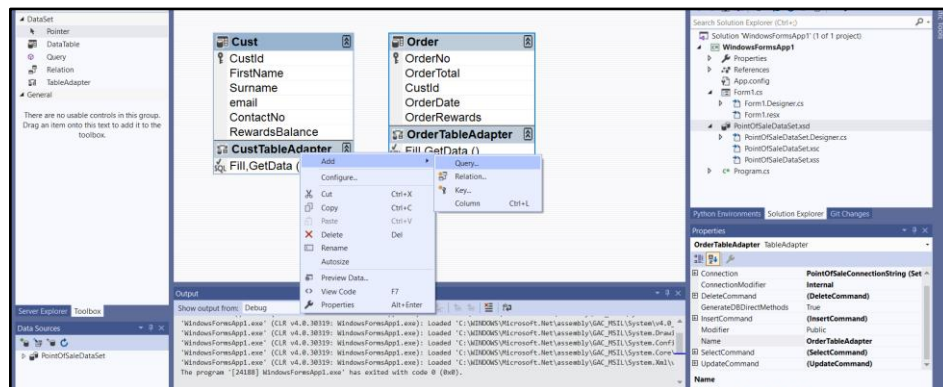


17. You are all set to go with the processing!

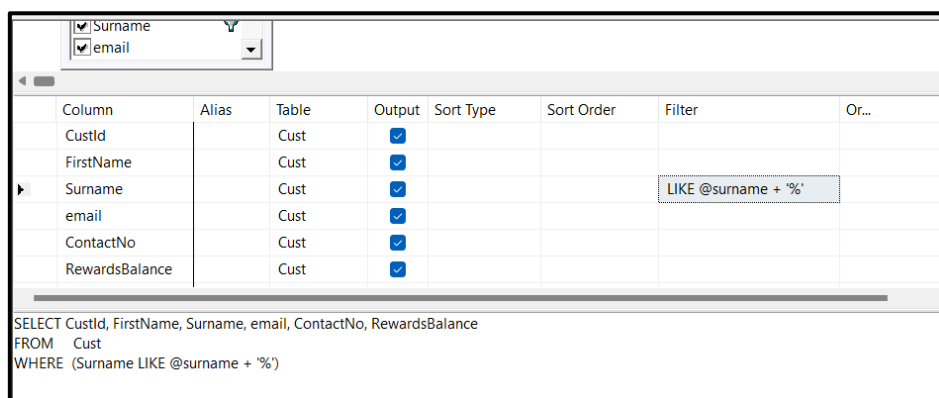
PART C

18. The main part of the processing at this stage is allow the end user to enter the first few letters of the customer name and the system fills up the gridview with all the possible names that will fit that criterion. In order to achieve this effect, as soon as a letter is added to the textbox, the gridview becomes populated with all surnames that are **like** the letters that are entered. The event that triggers the generation of data for the gridview is the *change of text* in the textbox

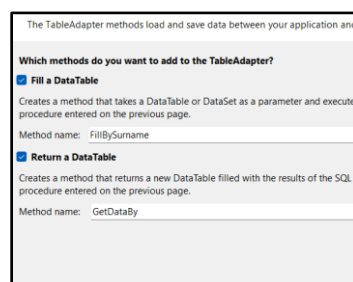
The trick here is to add a new method onto the customer table adapter. In the dataset class, right click the Customer table adapter and add a new query



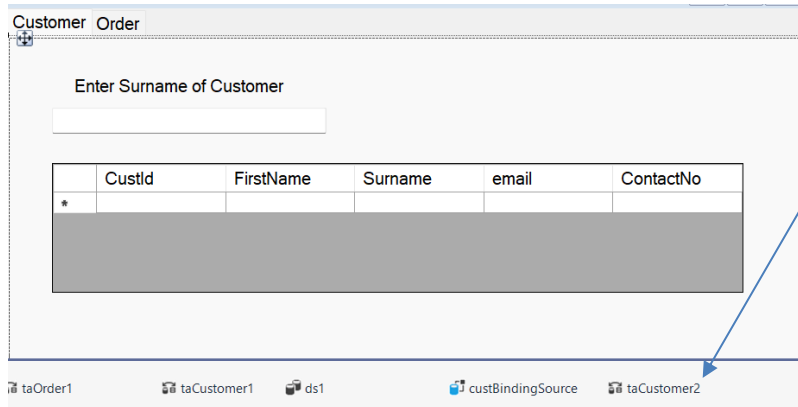
Get to the query builder and change the sql statement to look like...



Save this new query as a method on the table adapter and re-build the application



19. Now that the Customer table adapter class has changed in configuration, drag another table adapter object for the customer onto the form and use this instead of the original customer table adapter object (this is quite a nuisance, but just go with it for now!)



20. The code that follows simply fills the gridview with the correct customer based on the input in the textbox

```
private void txtCustSearch_TextChanged(object sender, EventArgs e)
{
    taCustomer2.FillBySurname(ds1.Cust, txtCustSearch.Text);
}
```

21. For example, when you type a letter like “c”, the response is...

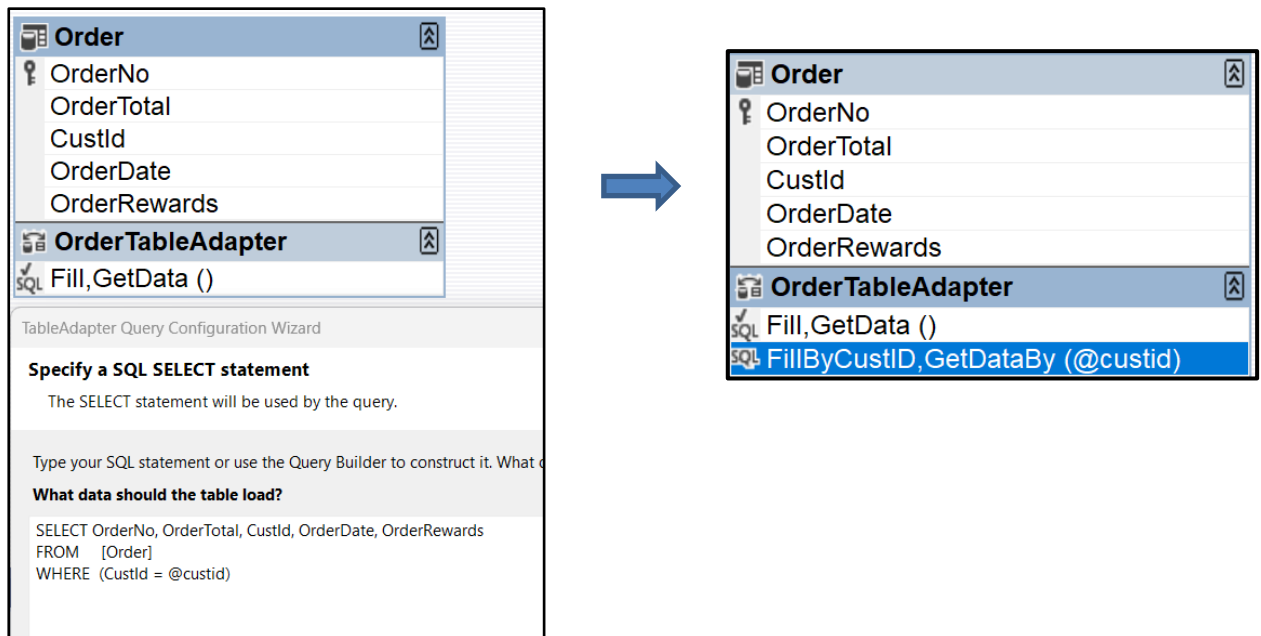
Enter Surname of Customer

c

	CustId	FirstName	Surname	email	ContactNo
▶	2	Miley	Cyrus	cyrus@gma...	23456
	3	Black	Coffee	coffee@ya...	32456
	7	Pryankra	Chopra	chopra@g...	32467

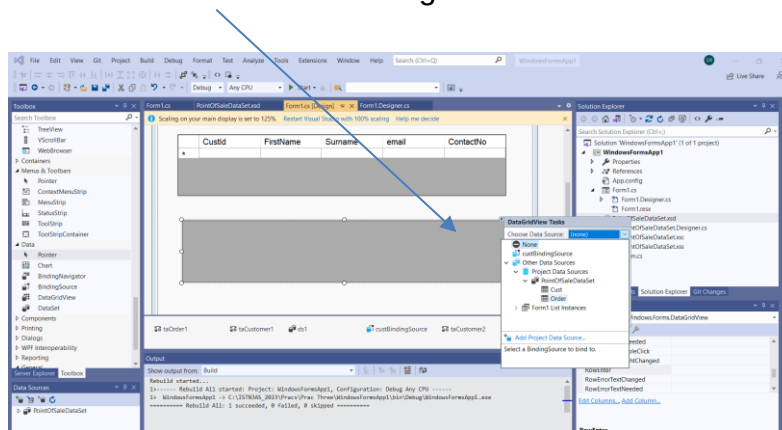
22. The next phase is to display the list of transactions for each selected row. To do this add another gridview that will display the orders of only the customer that is selected/active in the gridview.

23. The next step is to prepare the interface...Add another gridview, and configure the Order table to contain a new query that fills the dataset only with the orders of a specific customer. To do this get to the Query Builder of the Order table in the dataset class and create a new query that looks like...



Rebuild the application!

24. Set the Data source for the 2nd gridview



25. The idea is that as soon as a row is selected in the Customer gridview, the details of the customer order is displayed in the 2nd gridview. To achieve this effect, you have to find the correct event handler...

26. The RowHeaderMouseClicked is the event handler that will do the trick...

Enter the Customer Surname

	CustId	FirstName	Surname	email	ContactNo
▶	2	Miley	Cyrus	cyrus@gm...	23456
	3	Black	Coffee	coffee@ya...	32456
	7	Pryankra	Chopra	chopra@g...	32467
	9	Jimmy	Carr	carr@yaho...	22113
	32	Charlie	Chaplin	chaplinchar...	44232

Customer ID:2

R2 097,00

	OrderNo	OrderTotal	OrderDate
	2	R456,00	2026/03/14
	61	R44,00	2023/04/23
	66	R26,00	2023/04/23
	67	R57,00	2023/04/23
	84	R60,00	2023/05/10
▶	89	R39,00	2023/05/13

```
private void gvCustomer_RowHeaderMouseClicked(object sender, DataGridViewCellEventArgs e)
{
    taOrder2.FillByCustID(ds1.Order, (int)gvCustomer.CurrentRow.Cells[0].Value);
}
```

*****END OF PRAC 3*****